

Hippie: A Data-Paralleled Pipeline Approach to Improve Memory-Efficiency and Scalability for Large DNN Training

Xiangyu Ye, Zhiquan Lai, Shengwei Li, Lei Cai, Ding Sun, Linbo Qiao, Dongsheng Li

National Key Laboratory of Parallel and Distributed Processing
Computer College, National University of Defence Technology, China
{xyye,zqlai,lucaslee,cailei,sunding,qiao.linbo,dslj}@nudt.edu.cn

ABSTRACT

With the increase of both data and parameter volume, it has become a big challenge to efficiently train large-scale DNN models on distributed platforms. Ordinary parallelism modes, i.e., data parallelism, model parallelism and pipeline parallelism, can no longer satisfy the efficient scaling of large DNN model training on multiple nodes. Meanwhile, the problem of too much memory consumption seriously restricts GPU computing efficiency and training throughput. In this paper, we propose *Hippie*, a hybrid parallel training framework that integrates pipeline parallelism and data parallelism to improve the memory efficiency and scalability of large DNN training. *Hippie* adopts a hybrid parallel method based on hiding gradient communication, which improves the throughput and scalability of training. Meanwhile, *Hippie* introduces the last-stage pipeline scheduling and recomputation for specific layers to effectively reduce the memory overhead and ease the difficulties of training large DNN models on memory-constrained devices. To achieve a more reasonable evaluation of the optimization effect, we propose an index of *memory efficiency* (ME) to represent the tradeoff between throughput and memory overhead. We implement *Hippie* based on PyTorch and NCCL. Experiments on various models show that *Hippie* achieves above 90% scaling efficiency on a 16-GPU platform. Moreover, *Hippie* increases throughput by up to 80% while saving 57% of memory overhead, achieving 4.18× memory efficiency.

CCS CONCEPTS

• **Computing methodologies** → **Parallel computing methodologies**; **Distributed computing methodologies**; **Neural networks**.

KEYWORDS

deep learning, hybrid parallelism, distributed training, memory efficiency

ACM Reference Format:

Xiangyu Ye, Zhiquan Lai, Shengwei Li, Lei Cai, Ding Sun, Linbo Qiao, Dongsheng Li. 2021. Hippie: A Data-Paralleled Pipeline Approach to Improve Memory-Efficiency and Scalability for Large DNN Training. In *50th International Conference on Parallel Processing (ICPP '21)*, August 9–12, 2021, Lemont, IL, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

ICPP '21, August 9–12, 2021, Lemont, IL, USA

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-9068-2/21/08...\$15.00

<https://doi.org/10.1145/3472456.3472497>

Lemont, IL, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3472456.3472497>

1 INTRODUCTION

Recently, in pursuit of higher performance, the scale of deep neural network (DNN) models has been developing toward to deeper and larger, which has greatly exceeded the memory and computing capability of a single accelerator (e.g., GPU). Many of the latest models, such as the GPT series [4, 37, 38] and recommendation system [9, 44] have billions of parameters. With the growing memory requirement and computation of the large model training, it is very necessary to apply parallel training on distributed platforms to accelerate the training process.

Various parallelism modes, e.g., data parallelism (DP) [11, 26, 42], model parallelism (MP) [12, 22] and pipelined model parallelism (PMP) [19, 30, 33, 47, 48, 50], have been proposed to accelerate the DNN training. In DP, each worker trains a copy of entire model with sharded training data, and synchronizes the gradients among all workers periodically. DP is the most common used parallelism mode as it's easy to implement and load balancing when the GPU memory is enough. However, DP is not suitable for large-scale models due to that the high memory requirements could easily exceed the GPU memory limits. Asynchronous PMP such as PipeDream [30] comes to the same problem because multiple copies of weights will be generated during training.

MP divides models to ease the problem of too much memory consumption on memory-limited devices. Avoiding multiple copies of weights, synchronous PMP [19] improves throughput and reduces memory to a certain extent. However, difficulties in partitioning the model prevent these approaches from being applied on more nodes. Hybrid parallel approaches [13, 31] improve the convenience of scaling, and some of them further reduce the memory footprint. Unfortunately, similar to DP, gradients synchronization overhead still becomes the major factor preventing linear scalability.

Although much effort [14, 20, 23, 45, 49] on large DNN training, the fact is that few approaches can meet the requirements of low memory overhead and high scalability at the same time. In this paper, we propose *Hippie*, a distributed training scheme that integrates PMP and DP. The purpose of integrating PMP and DP is to combine the advantages, as PMP performs well on fewer nodes and DP makes for easy scaling. Moreover, *Hippie* hides the gradient communication and optimizes memory overhead to address problems existing in scalability and memory overhead, respectively. We evaluated *Hippie* on various benchmarks, including image classification and machine translation tasks. Experiments show that *Hippie* can consistently guarantee scalability and improve memory

efficiency under the premise of maintaining the model convergence. Besides, *Hippie* performs better for larger models.

The contributions of our work are summarized as follows:

1. A Scalable Hybrid Parallel Approach. We feature a novel *Communication Schedule* to hide the gradient communications and reduce idle time. This improves the scalability and facilitates distributed training in large clusters. For the convenience of expansion, *Hippie* employs fewer GPUs to form a single pipeline worker and carries out training on multiple nodes by replicating workers. Thus we could fit models effortlessly to additional GPUs without repartitioning it.

2. Approaches to Optimize Memory Efficiency. We introduce *Last-stage Schedule* and *PMP Planner* to reduce memory overhead. The *Last-stage Schedule* rearranges the computation sequence to releases the graph in advance, relieving the memory pressure. *PMP Planner* is designed to identify specific layers with low memory efficiency and apply recomputation [6, 17]. After this, it figures out a pipeline scheme with the balanced model partitioning.

3. Performance Evaluation. We implement *Hippie* based on PyTorch and NCCL and conduct experimental evaluation on various DNN models. The results show that *Hippie* achieves better scaling efficiency than the general hybrid approach combined Gpipe and *DP*. Moreover, *Hippie* gets up to 4.18 $\times$ memory efficiency than *DP*.

2 BACKGROUND AND MOTIVATION

2.1 Basic Parallelism Approaches

Data parallelism (*DP*) and model parallelism (*MP*) [8] are common methods for DNN training. In *DP*, each worker keeps a model copy and holds a subset of the dataset. To speed up training, communication pace and topology related approaches [15, 29] are developed constantly. Asynchronous Parallel (ASP) [32] and Stale Synchronous Parallel (SSP) [10, 18] reduce the restriction on consistency, allows asynchronous updates without affecting convergence. Besides, decentralized collective [16, 27, 28] operations like AllReduce performs better than classic parameter servers [26], has been widely used in modern deep learning frameworks [3, 36, 42].

While *MP* [5, 12, 22, 25] divides model layers into multiple parts and places them on multiple workers, and same to the forward and backward computation, making this a solution for training large-scale models. But limited by the inevitable data dependency, only one worker could perform computation at any moment in *MP*, resulting in low computational efficiency.

Based on model partition in *MP*, *PMP* splits inputs into several mini-batches and injects them into DNNs continuously. Without the dependency between mini-batches, data can be utilized in parallel as a pipeline. *PMP* methods can be separated into two categories according to the different update rules: asynchronous pipelines and synchronous pipelines. The representational asynchronous pipeline, Pipedream [30] strived for the highest throughput and resource utilization, updating model weights as soon as the backward computing is completed, but leading to multiple model parameter versions, which worsen the memory overhead problems. The iconic synchronous pipeline Gpipe [19] paid more attention to the memory, and the model is updated synchronously by grouped mini-batches. However, synchronous updating will lead to bubbles inside each iteration, restricting the training efficiency. Although pipeline

parallelism is efficient for training on a small number of nodes, it is not the case for training on large-scale nodes due to the increasing memory overhead, transmission speed loss, and other problems.

2.2 Hybrid parallelism

Hybrid parallelism combines the above parallel strategies to seek a more comprehensive parallel training scheme. Alex Krizhevsky manually partitioned AlexNet, applied the *MP* to linear layers with many parameters but few outputs, and *DP* to convolutional layers with few parameters but large output, which is an early attempt of hybrid parallelism [24]. FlexFlow [21] proposed a new search space SOAP for DNN parallelization strategy to seek the optimal hybrid scheme for *MP* and *DP* from four dimensions: sample, operator, attribute, and parameter. Due to the distinct layers in models, the single parallel method cannot achieve the optimal performance, and hybrid parallelism is a whole new ball game [7, 34].

With the introduction of pipeline parallelism, researchers have integrated *PMP* and *DP*. HetPipe [35] combined parameter server with *PMP* and explored hybrid parallelism under heterogeneous architecture to improve the efficiency of DNN training. The pipeline part and parameter server part of Hetpipe both adopted the idea of asynchronous update, and too much stale data will inevitably affect the convergence of the model. Multiple versions of model parameters stored in memory increase memory overhead as well.

DAPPLE [13] started from the synchronous pipeline and brought *DP* to their implementation. As a pipelined *DP* approach, DAPPLE performed *DP* in intra-nodes and *PMP* in inter-nodes, which is difficult to scale. DAPPLE adjusted calculation order of mini-batches in Gpipe, releasing the graph in advance and relieving memory pressure to a certain extent. However, there are both bubbles existing in pipeline and communication overhead coming from AllReduce, restricting the efficiency of DAPPLE. Moreover, with the high-speed intra-connection technology such as NVLink [2] within nodes and low bandwidth network between nodes, the bottleneck of *DP* lies in the inter-nodes bandwidth. DAPPLE could make full use of the accelerations of NVLink, and shown the best performance compared with *DP*, which also indicates that the DAPPLE has some limitations.

2.3 Motivation

For better performance in large DNN training, there are two challenges we have to address. The first challenge is to guarantee high scalability, optimizing training on large-scale nodes. The second challenge is to reduce memory overhead while maintaining decent throughput, improving support for large-scale DNN models. There is still few work to optimize both at the same time.

For a more reasonable evaluation of the optimization, we measure the memory efficiency of accelerators in terms of both throughput and memory overhead, which is distinctive feature. Most of the existing parallel approaches share the characteristics that throughput and memory consumption increase and decrease simultaneously. For example, Pipedream increases throughput as well as memory footprint. *MP* and Gpipe reduce the memory overhead but also the throughput. As shown in the following formula, we define a new index to measure the efficiency of the parallel approach: the ratio of the average throughput per device after expansion to

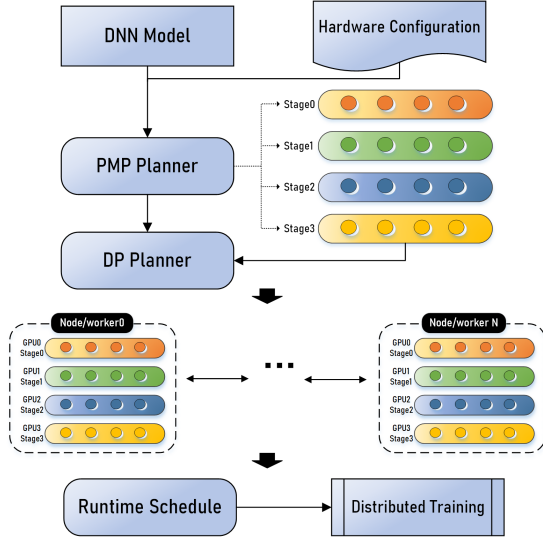


Figure 1: High-level workflow of *Hippie*

the maximum memory consumption per device (GB) is defined as memory efficiency.

$$ME = \frac{Scalability \cdot Throughput}{Memory}$$

Higher throughput with a lower memory footprint can be called efficient use of memory, and lower throughput with a higher memory footprint can be called inefficient use of memory. Therefore, for large-scale models, high scalability with a low memory footprint is the key to maintain memory efficiency. Our research will focus on three aspects: improving scalability, reducing memory footprint, and balancing the partitioned model.

3 THE HIPPIE APPROACH

Overview: Fig. 1 shows a high-level workflow of *Hippie*, which mainly consists of *Planner* and *Runtime Schedule*. The planner includes a *PMP Planner* and a *DP Planner*. The *PMP Planner* picks the layers that need to be recomputed and divides the model layers to generate a balanced pipeline parallelism strategy called a *PMP worker*. And the *DP Planner* replicates the *PMP worker* and extends it to multiple nodes. *Runtime Schedule* is responsible for scheduling during training, comprising communication scheduling and last-stage scheduling, ensuring both order and efficiency of *Hippie* training operations.

A naive implementation of integrating *PMP* and *DP* contains two aspects. First, stages in the same worker are running as a usual pipeline. Besides, stages in the same location of workers synchronize gradient through AllReduce after every iteration. However, to realize efficient distributed training by integrating *PMP* and *DP*, *Hippie* has to overcome two of the challenges mentioned in Section 2.3. In *Hippie*, we introduce the *Communication Schedule* to address the first challenge, the *Last-stage Schedule*, and the *PMP planner* to address the second, which are discussed in the following sections.

3.1 Communication Schedule

Hippie treats a single pipeline as a worker, and it is convenient to layout *Hippie* on more nodes by copying workers. Each intra-worker stage cooperates by transferring intermediate data of layers. Furthermore, the same inter-worker stages will be divided into a group for gradient averaging to maintain synchronization. During an iteration, the accelerators mainly have five states: forward computations, backward computations, gradient communication, parameter update, and idle status. Planning the execution of the pipeline and reasonably arranging these states are the keys to improving throughput and scalability. The proportion of communication time and idle time should be reduced as much as possible.

As shown in Fig. 2, differed from the order that communications after computations in the usual *DP*, *Hippie* advances the communication operations and covers it within the process of computations. Since each mini-batch’s forward and backward computations will be divided into multiple processes and spawned to several devices, the timeline of training is usually presented as a trapezoidal shape. Fig. 2 shows a four stages *Hippie* example with well-designed communication scheduling. We use blue and green blocks to indicate forward and backward computations and orange blocks to indicate gradient communication and updates. For the stages in the former part, the idle time between forward computations and backward computations of intra-step is the longest. Thus we arrange the communication and averaging of gradients here. For stages in the latter part, the longest idle time locates between backward computations and forward computations of inter-step. Therefore, their gradients synchronization can be done in the usual order. Fig. 2 has marked the period for each stage to communicate through the AllReduce operation. Since the gradient communication in the former part of the pipeline is advanced, one more process – exchanging gradient is needed before the update. Specifically, the newly calculated gradients are exchanged with the averaged gradients. The averaged gradients are used for updating, while the new gradients are used for AllReduce in the next step. After these scheduling, *Hippie* hides all the operations of gradient communication during the computation process, with almost no delay between each step, which improves scalability greatly.

Assuming that the model is perfectly divided according to the execution time of layers, the time taken for each stage-computation is the same. We can discuss the feasibility that the gradient averaging operation can be covered. As shown in the Fig. 2, for *Hippie* with M -stages, the shortest time available for AllReduce average gradient operation is $M/2 \cdot (T_{forward} + T_{backward})$. For example, the available time of stage1 and stage2 in Fig. 2 is $2 \cdot (T_{forward} + T_{backward})$. For layers in charge of a stage, if the $T_{computation} : T_{communication} > 2/M$, then the gradient communications of this stage can be covered by computations. When M is 4, the $2/M$ is $1/2$, which means if the communication time is twice as long as the computation time, the communication time can still be hidden with high tolerance. The more stages the *Hippie*’s pipeline has, the easier gradient communications are to be hidden.

3.2 Last-stage Schedule

In *Hippie*, the memory overhead mainly consists of three parts: model parameters, intermediate results of the graph, and boundary

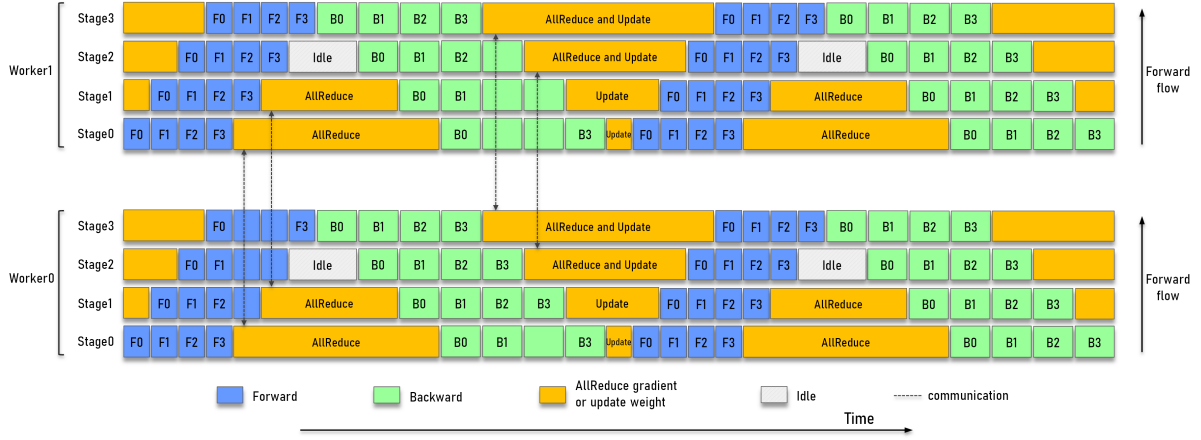


Figure 2: Hybrid parallelism approach with Communication Schedule. The numbers in cells represent intra-step mini-batch IDs. Each pipeline forms a single worker. Stages of the same location in workers synchronize gradient through AllReduce. After scheduling, the AllReduce overhead originally between steps has been hidden by computations.

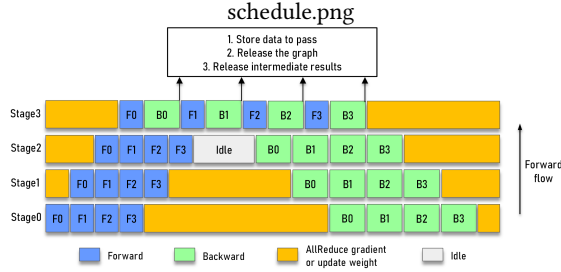


Figure 3: A Last-stage Schedule example

data between stages. The first two types of data are common in training, and the third is the additional memory footprint when a model is partitioned. In the asynchronous pipeline, the backward computations will not be carried out until forward computations of multiple mini-batches complete. Therefore, the intermediate results and boundary data of graphs need to be stored in the memory. This will significantly increase the memory consumption for the layers with relatively large output. We introduce two ways to reduce memory overhead: the *Last-stage Schedule* and the recomputation for specific layers. The details of recomputation will be presented in Section 3.3.

In the synchronous pipeline, the same number of graphs as mini-batches will be accumulated while reaching the peak memory consumption. Whenever a backward computation is completed, and the gradient required for the previous layer is passed, the corresponding graph can be released. The last stage may become the largest memory consumption, thus becoming the bottleneck of training. For example, in language models, the output dimension of the last layer has to match the dimension of dense embedding, usually tens of thousands of dimensions. In order to avoid excessive memory consumption of the last stage, *Hippie* schedules one forward of mini-batch followed by one backward strictly, advancing the release of the graph. The particularity of pipeline training is

that the data of forwards and backwards flow in opposite directions, making the last stage act as the last forward, also the beginning of backward. Therefore, the scheduling above will not affect the other stages except the last. Fig. 3 shows the specific workflow of the last stage. Forward computations are performed, and then the backward computations immediately. After saving the data that needs to be transferred, the graphs and the intermediate results of this mini-batch can be released, and the same for the next mini-batch.

3.3 Pipeline Plan

In general, the optimization goal of approaches such as Pipedream and DAPPLE is to maximize throughput. Nevertheless, we take memory footprint into account particularly, our optimization goal becomes memory efficiency, which contains both throughput and memory footprint. As the recomputation affects not only memory overhead but also computation speed, the selection of specific layers applied recomputation and partitioning model algorithms are integrated as a whole. For *Hippie*, we take the averaged execution time (s) of a single global batch multiplied by the maximum memory consumption (GB) in all stages as our performance metric, referred to *cost*. The memory efficiency *ME* is calculated by $\frac{1}{cost}$. The execution time of stages includes computation time and communication time. Parameters and the data for computation make up the total memory consumption. All of those vary with the recomputation and model partition scheme, as shown in the formula below.

$$T_k = \sum_{i=0}^L \begin{cases} F_i + B_i & \text{no-recompute} \\ 2 \cdot F_i + B_i & \text{recompute} \end{cases} \quad (1)$$

$$T_{cal} = N \cdot T_{last} + \sum_{k=0}^{M-1} T_k \quad (2)$$

$$T_{comm} = T_{intra} + \begin{cases} 0 & T_{inter} < M \cdot T_{last}/2 \\ T_{inter} - \frac{M \cdot T_{last}}{2} & T_{inter} > M \cdot T_{last}/2 \end{cases} \quad (3)$$

For Formula 1, the F_l and B_l denote the time of forward computation and backward computation in a layer, respectively. The T_k refers to a single mini-batch execution time of a single stage with L layers. Execution time for layers applied recomputation is $2 \cdot T_{forward} + T_{backward}$, and $T_{forward} + T_{backward}$ for layers that don't. On this basis, T_{cal} is defined as all mini-batches execution time for all stages. As the timeline for the training of N mini-batches and M stages forms a trapezoidal shape, the T_{cal} is calculated from Formula 2, which means N times the execution time of the last stage plus the individual execution time of every previous stage.

The communication time T_{comm} includes the intra-pipeline and inter-pipeline communication overhead. Time for intra-pipeline communication T_{intra} refers to the overhead of passing data between stages, which can be measured easily. The time for inter-pipeline communication is optimized by *Hippie Communication Schedule* and is zero in most cases, as discussed in Section 3.1.

$$S_{mem} = \sum_{k=0}^L \left(S_{model} + \begin{cases} N \cdot S_{inter} & \text{no-recompute} \\ S_{inter} & \text{recompute} \end{cases} \right) \quad (4)$$

$$ME = \frac{1}{cost} = \frac{1}{(T_{cal} + T_{comm}) \cdot \max \lim_{0 \leq i \leq M} S_i} \quad (5)$$

Memory consumption for a single stage S_{mem} is the sum of parameters S_{model} and data for computation S_{inter} of all layers in this stage. The S_{model} is composed of weight, gradient, parameters in optimizer, and others, which are determined by the number of layers. The rest of consumption depends on the application of recomputation. For the iteration with n mini-batches, recomputation can reduce the dynamic memory consumption from $N \cdot S_{inter}$ to S_{inter} .

By combining Formula 1, 2, 3, 4, we can get an expression for ME , as shown in Formula 5. We need to find the maximum memory footprint $\max S_i$ from M stages and multiply it by the total execution time. Therefore, the optimal scheme that applies the recomputation and partitioning model is the optimal solution that maximizes ME .

4 IMPLEMENTATION

Overview: *Hippie* is implemented based on PyTorch and NCCL. The main operations of each stage in *Hippie* include forward computation, backward computation, intra-pipeline communication, and inter-pipeline communication. At the same time, each stage does different operations, so there is a scheduler to manage the stage to ensure orderly training. In this section, we mainly describe the communication groups, the operation of stages, the processing of boundary data, and the preservation of gradients in *Hippie*.

Communication groups: Communication groups: *Hippie* maps the model layers to the stage as described in Section 3.2. Each stage corresponds to a GPU and a process. Before training, *Hippie* will first group each stage. Each stage has two identities: a stage in the pipeline and a worker in the data parallelism. In the pipeline, each stage groups with neighboring stages to transfer forward computation results and gradients named intra-group. The stages at the same position of each pipeline will also be divided into groups called inter-group. Similar to data parallelism, inter-group gradients will be averaged through AllReduce.

Table 1: Models and datasets for evaluation

Model	# of Params	Dataset	Target Accuracy
GNMT-8	191M	WMT16 EN-De	24 BLEU
GNMT-16	290M	WMT16 EN-De	24 BLEU
VGG-16	138M	ImageNet	70% top-1
AmoebaNet-18	318M	ImageNet	70% top-1

Operation of stage: There are four types of stages in *Hippie* named as *stage 0*, *stage 1*, *stage 2*, *stage 3*. Of all the stages, only the *stage 0* has the “DataLoader” and injects new data into the pipeline. The training data of the rest stages are all obtained by communication. The working loop in *stage 0* is as follows: perform forward computations of n mini-batches → communicate with other stages in inter-group → perform backward computations of n mini-batches → store new gradients → update parameters with the gradients after AllReduce. Except for obtaining input data from communication transmission, the other steps of *stage 1* are the same as the *stage 0*. The *stage 2* performing in normal order: perform forward computations of n mini-batches → perform backward computations of n mini-batches → communicate with other stages in the inter-group → update. The *stage 3*, which is the last stage under the scheduling mentioned in 3.2, works as follows: perform forward computations and backward computation for n times → communicate with other stages in the inter-group → update.

Boundary data: Boundary data is the input and output data of each stage. According to the chain rule in back-propagation, the gradients of inputs have to be taken out and transmitted to the previous stage to continue backward computations. In PyTorch, leaf data in a graph such as the input do not save gradients by default, which needed to define that it needs gradient by registering hook explicitly. For stages exclude the last, the gradients obtained by the hook will be immediately passed to the next as soon as the local backward computations are completed. For the last stage, transmissions start from half of the step. The outputs of the local forward computations are stored in GPU memory, and its pointer is put in the work queue. These outputs will not release until the mini-batch completes its backward pass.

Preservation of gradients: For the stages applied *Communication Schedule*, an additional gradient should be stored in the gradient buffer. After each step, the newly calculated gradients will be exchanged with the data in buffers so that averaged gradients update the weights. Gradients in the buffer are still in dictionary order. Therefore, during AllReduce operations, the correspondence of data dimension can be guaranteed because multiple pipelines are copied.

5 EVALUATION

5.1 Experimental setup

Models and datasets: We evaluated the performance of *Hippie* among four diverse DNN models, including image classification and NLP tasks. As shown in Table 1, we trained VGG-16 [43] and AmoebaNet(18, 416) [39] with the ImageNet [40], GNMT-8 and

GNMT-16 [46] with the WMT16 En-De [41]. The AmoebaNet (L, D) denotes AmoebaNet-D model with L normal cell layers and filter size D, GNMT-N represents GNMT model contains N LSTM layers.

Hardware configuration: Our experiments are based on a familiar GPU cluster, using up to 4 server nodes. Every node is equipped with four NVIDIA GeForce RTX 3090 GPUs within 24GB memory each and two Intel Xeon 4214R @ 2.40GHz CPUs. All servers run 64-bit Ubuntu 18.04, CUDA 11.1, cuDNN 8.0.5, NCCL 2.7.8 [1], PyTorch 1.8 and connected by 100 Gbps InfiniBand.

Batch sizes and training methodology: We tried various global batch size (GBS) to explore the GPU memory overhead, throughput, and scalability of our approach. The computation time of models varies from different batch sizes, while the communication traffic remains the same. In our experiments, for VGG and GNMT, there are four GBS groups of 256, 512, 1024, and 2048, while GBS 32, 64, 128, and 256 for AmoebaNet requiring larger memory. Based on this, we could track the memory efficiency and scalability of each method with multifarious computation communication ratios to verify the universality of *Hippie*.

Altogether four approaches are involved in our experiments: (1) **Hippie**: The Hippie with four stages and with two stages within single nodes performs better as efficiency will be reduced greatly by the cross-node pipeline. (2) **Gpipe+DP**: Since Gpipe does not support data parallelism and there are 16 GPUs, models must be divided into 16 parts, which is inoperable. We had to implement a training process that integrates Gpipe and DP, without applying recomputation and hiding communication. Overall there are four stages in Gpipe for our experiments. (3) **DP**: Data parallelism with intra-iteration computation communication overlap, which is one of the most efficient distributed training approaches under the PyTorch framework. (4) **MP+DP**: The method implemented to integrate MP and DP for training larger models.

The Hyperparameters are consistent with common practices of the ML community. We applied the Adam optimizer to train GNMT, the SGD optimizer for VGG and AmoebaNet. The initial learning rates for GNMT-8, GNMT-16, VGG-16, AmoebaNet-18 are 0.001, 0.001, 0.01, 0.1 respectively. The target validation accuracy is as follows: top-1 accuracy of 70% for image classification and BLEU score of 24 for GNMT. During the training epochs, we properly adjusted the learning rate according to prior works and applied the learning rate warmup to accelerate the convergence in the demands of models. All of the training methods in the same experiments set used the same learning rate schedules and performed full (fp32) precision.

5.2 Planning results

Table 2 shows the setup of *Hippie* trained by 16 GPUs, determined by *PMP Planner*. The training plan refers to the topology of training. Such as x stages- y DP represents x stages in *Hippie*'s pipeline, and each pipeline will be copied y times to make use of 16 GPUs. Split Position corresponds to the number of layers contained in one stage. And Recompute means the layers would be recomputed to optimize memory overhead.

In both GNMT models, the memory overhead and execution time of layers are relatively balanced, so the model is divided into four stages. In terms of image classification models, VGG-16 and

Table 2: Planning results

Model	Training Plan (16GPUs)	Split Position	Recompute
GNMT-8	4Stages-4DP	4:3:3:1	Attention layers
GNMT-16	4Stages-4DP	5:6:5:4	Attention layers
VGG-16	2Stages-8DP	6:14	All layers in first stage
AmoebaNet-18	2Stages-8DP	9:15	All layers in first stage

AmoebaNet-18 have the following characteristics: with Pool Layers, images are subsampled every few layers from the input model, reducing memory overhead gradually. Therefore, the memory activation of each layer in the model is remarkably uneven, and the first few layers would dominate the memory overhead. Under the 4 Stages architecture, the memory footprint of each stage is relatively high, and the throughput is moderate. Hence, we selected the 2Stages-8DP structure with higher memory efficiency and applied recomputation at all layers in the first stage. In summary, we compared the performance of *Hippie*, *Gpipe+DP*, *DP* on GNMT-8, GNMT-16, and VGG-16. Owing to the large memory footprint, we replaced *DP* with *MP+DP* on AmoebaNet-18.

5.3 Memory efficiency

Fig. 4 shows performance for all models with fixed number of GPUs=16. With the contribution of the *Communication Schedule*, *Hippie* maintains a decent throughput. Meanwhile, as shown in the first column of Fig.4, *Hippie* could keep the memory footprint to a sufficiently low value for each GBS configuration, only half of *DP* in some cases. Although sometimes recomputation at the expense of reducing throughput, the memory efficiency, one of our major optimization goals, is significantly improved. Moreover, the memory efficiency of *Hippie* is always the highest among the above three methods. We will present details in different models below.

GNMT16/GNMT8: Due to the existence of Embedding layers, GNMT models hold a large number of parameters, and the computation cost of the LSTM layer is relatively small. Thereby the proportion of communication overhead of GNMT is the highest among all models in our experiments. The throughput of *Hippie* exceeded data parallelism by 32%, 42%, 19% in GNMT-8, and 80%, 55%, 23% in GNMT-16, corresponding respectively to 256, 512, 1024 GBS. As for GBS 2048, along with the longer computation time, more communication overhead can be hidden via overlap in *DP*. Thus throughput of *DP* is slightly higher than *Hippie*. However, the peak memory overhead is reduced significantly by *Hippie* in this scenario. Furthermore, the memory footprint maintained by *Hippie* at 40%-60% of data parallelism regardless of the GBS. For less memory overhead approach *Gpipe+DP*, *Hippie* still drops memory footprint by around 20%. Altogether, when GBS ranged from 256 to 2048, *Hippie* could get up to 2.65× memory efficiency in GNMT-8, and 4.18× in GNMT-16. Due to the fact that there is no scheme

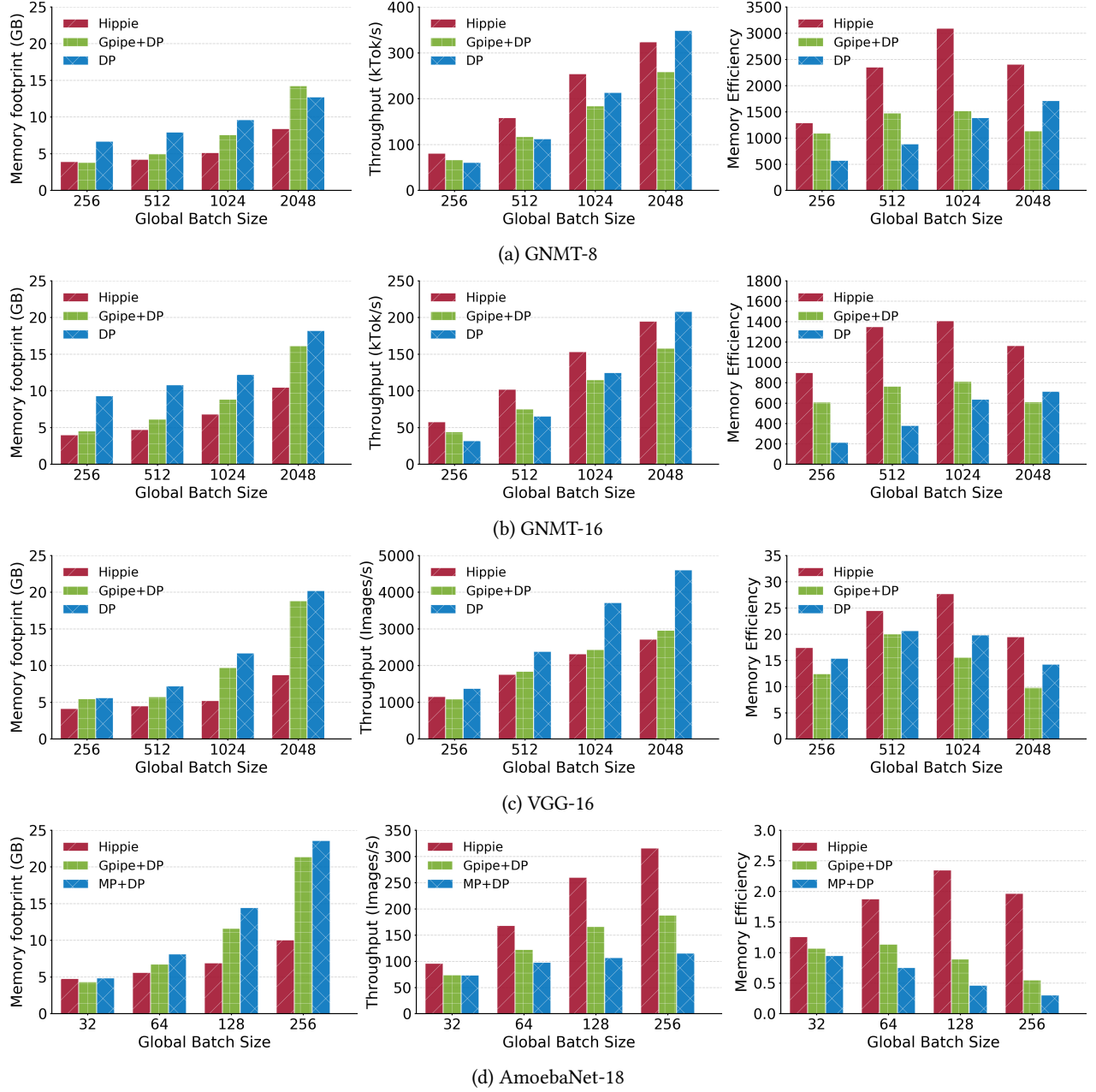


Figure 4: Performance comparison of four models using 16 GPUs. Various global batch sizes are applied to evaluate memory footprint, throughput and memory efficiency of different approaches.

to hide the communication and optimize the memory overhead in *Gpipe+DP*, its memory efficiency falls in between *Hippie* and *DP*.

VGG-16: The model size of VGG-16 is slightly smaller than GNMT-16, while the convolutional layers take longer computation time, so the proportion of communication overhead is significantly reduced compared with GNMT. And in VGG-16, the convolutional layers have a high computation cost and a small communication cost, while the fully connected layers are opposite. Therefore, most

communication operations could be covered through *DP* overlap during training VGG-16, resulting in high throughput. But *DP* consumes 1.3×, 1.6×, 2.2× and 2.3× peak memory, and finally achieves 0.88×, 0.84×, 0.72× and 0.73× memory efficiency respectively compared to *Hippie*. Similar to GNMT, the performance of *Gpipe+DP* approach is secondary in both memory overhead and throughput. But the memory efficiency of *Hippie* is up to 98% better than *Gpipe+DP*.

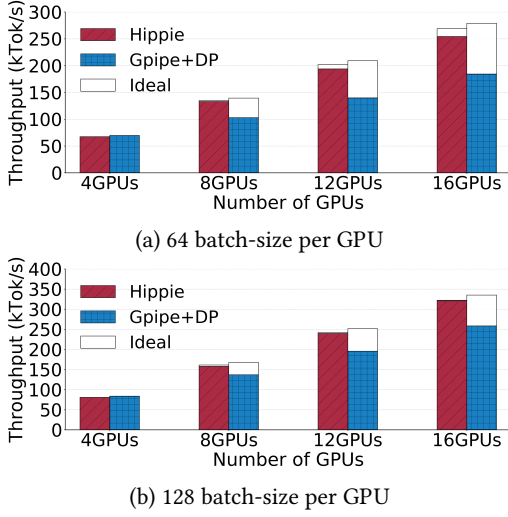


Figure 5: Multi-GPU scaling performance for GNMT-8

AmoebaNet-18: Limited by the device memory, *DP* is not available in AmoebaNet. We have to divide AmoebaNet into several parts and then combine *Gpipe* or *MP* with *DP* for training. As Fig. 4 shows, *Hippie* performs significantly better than the other two approaches. Although the *Gpipe+DP* approach has outperformed *MP+DP*, *Hippie* can decrease memory overhead by 20%-55% meanwhile increase throughput by 30%-68%. Surprisingly, when GBS=128 or GBS=256, *Hippie* outperforms *Gpipe+DP* with 2.64× and 3.56× memory efficiency. This improvement enables *Hippie* to perform pipeline with 2stages-8DP, which is more efficient than the pipeline with 4stages-4DP in *Gpipe+DP*. And the *Communication Schedule* further improves *Hippie*'s throughput and memory efficiency. It can be seen that *Hippie* has more outstanding training efficiency for large-scale models, especially those cannot be trained by a single GPU. *Hippie* is capable of limit the maximum memory consumption of AmoebaNet training at GBS=256 to 10GB with an impressive speed, allowing us to arrange the training in wimpy GPUs such as 1080Ti, which greatly reduces the research cost.

5.4 Scalability

In this section, we carry out the comparison of scalability with fixed batch size per GPU. The scalability of *Hippie* has significantly improved thanks to hiding communication. The GNMT, which has the highest proportion of communication overhead, is selected for this experiment. We compare the scalability of *Hippie* and *Gpipe+DP* on 4, 8, 12, and 16 GPUs. The average batch size of each GPU is set to 64 and 128, commonly used in GNMT training. We use the throughput of 4 GPUs as the benchmark, then the throughput of 16 GPUs in linear scaling should be four times. Fig.5 and Fig. 6 show the throughput of both approaches and compare it to ideal linear scaling results. Due to space, we did not show the scalability chart of other models. In fact, VGG-16 and Amoebanet-18 have better scalability because of the lower proportion of communication.

Multi-GPU scaling performance for GNMT-8 is shown in Fig.5. With 64 batch-size, scaled from 4 GPUs to 16 GPUs, *Hippie* gets up

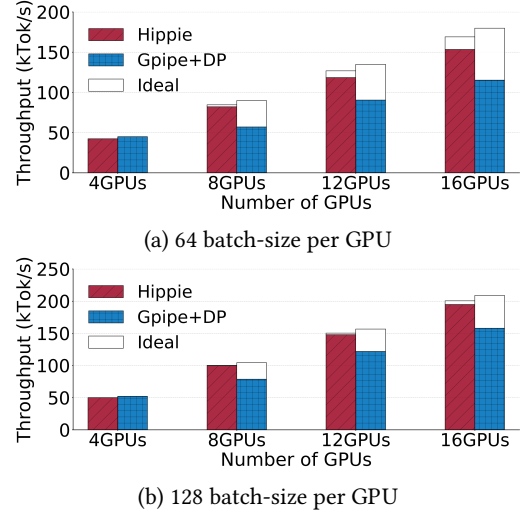


Figure 6: Multi-GPU scaling performance for GNMT-16

to 3.78× throughput while *Gpipe+DP* gets 2.64×. With 128 batch-size, the improvement is 3.98× in *Hippie* while 3.08× in *Gpipe+DP*. For GNMT-16 shown in Fig.6, when scaled to 16GPUs, *Hippie* gets 3.63× throughput while *Gpipe+DP* gets 2.56× with 64 batch-size. And *Hippie* gets 3.88× throughput while *Gpipe+DP* gets 3.02× with 128 batch-size.

Communication Schedule is proved to bring a significant increase in scalability, which makes *Hippie* outperform other hybrid parallel approaches vastly. Unfortunately, due to the limitation of the number of devices, we did not test on more GPUs. We believe that with the increasing number of GPUs, *Hippie*'s robust scalability could be more prominent. On the other hand, unlike most hybrid parallel methods, *Hippie* only needs to copy the same pipeline to more nodes after determining the model partition and pipeline scheme, making the implementation more convenient.

5.5 Convergence

Fig.7 shows the convergence of *Hippie*. We traced the BLEU score for GNMT-16 and accuracy values for VGG-16. The training curves of other models will not be demonstrated due to the space limit. From our observation, *Hippie* can reach target accuracy in a closing number of epochs to *DP*, confirming that *Communication Schedule* in Section 3.1 would not affect the convergence. We will demonstrate this theoretically below.

Under the parallel communication scheduling strategy, the parameters update rules can be summarized as synchronous intra-step updates and asynchronous inter-step updates. Synchronous intra-step update means that the updates of mini-batches within a step are kept in sync to ensure that multiple model weights will not be created. The asynchronous inter-step update specifically refers to the update strategy for the latter part of the pipeline, as discussed in Section 3.1. *Hippie* updates weights with the averaged gradients of the previous steps in these stages.

We denote w_l by weights of stages in the former part, which performs synchronous updates. And w_f for weights of stages in the

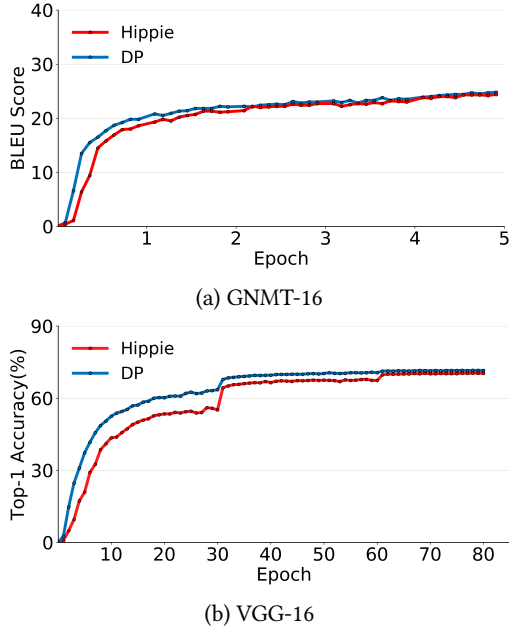


Figure 7: Accuracy vs. epoch using 16 GPUs

latter part, which perform asynchronous updates. The gradients calculated by the t -th global batch are expressed by $grad^{(t)}$. Under regular synchronous updating scheme, weight parameter w_l after n mini-batches with the initial values of w_{l-init} can be expressed as follows:

$$w_l = w_{l-init} - v \cdot (grad^{(1)} - grad^{(2)} - grad^{(3)} - \dots - grad^{(t)})$$

Under regular asynchronous updating scheme, weight parameter w_f can be expressed as follows:

$$w_f = w_{f-init} - v \cdot (0 - grad^{(1)} - grad^{(2)} - \dots - grad^{(t-1)})$$

Compared with w_l , w_f does not carry out parameter updates at the first time, while the gradients of last update are $grad^{(t-1)}$, which are one step less than w_l . Although w_l are slightly different from w_f in this updating formula, the training effect is the same. We can view the whole $(w_{l-init} - v \cdot grad^{(1)})$ as w_{l-init} , then the update of w_l will be synchronized with w_f , which is semantically similar to synchronously update.

6 CONCLUSION

In this paper, we present a distributed training framework for large-scale DNN models, *Hippie*, which integrates pipelined model parallelism with data parallelism. To address the challenges of large DNN training, *Hippie* introduces the *Communication Schedule* to reduce communication overhead and improve the scalability in large-scale computing nodes. In addition, the *Last-stage Schedule* and *PMP Planner* of *Hippie* significantly optimize the memory footprint to ease the training difficulties of large DNN models on memory-limited devices. Experimental results show that *Communication Schedule*

enables *Hippie* to maintain 90% scaling efficiency on a 16-GPU platform. The *Last-stage Schedule* and *PMP Planner* save 30%-60% memory consumption. Therefore, *Hippie* outperforms *DP* by up to 4.18× memory efficiency.

ACKNOWLEDGMENTS

This work is sponsored in part by the National Key Research and Development Program of China under Grant No.2018YFB0204300, and the National Natural Science Foundation of China under Grant No.62025208. Dongsheng Li is the corresponding author of this paper.

REFERENCES

- [1] 2019. NCCL. <https://developer.nvidia.com/nccl>
- [2] 2019. NVLink. <https://www.nvidia.com/en-us/data-center/nvlink/>
- [3] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. 2015. TensorFlow: Large-scale machine learning on heterogeneous systems.
- [4] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* (2020).
- [5] Chi-Chung Chen, Chia-Lin Yang, and Hsiang-Yun Cheng. 2018. Efficient and robust parallel dnn training through model parallelism on multi-gpu platform. *arXiv preprint arXiv:1809.02839* (2018).
- [6] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174* (2016).
- [7] Trishul Chilimbi, Yutaka Suzue, Johnson Apacible, and Karthik Kalyanaraman. 2014. Project adam: Building an efficient and scalable deep learning training system. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. 571–582.
- [8] Adam Coates, Brody Huval, Tao Wang, David Wu, Bryan Catanzaro, and Ng Andrew. 2013. Deep learning with COTS HPC systems. In *International conference on machine learning*. PMLR, 1337–1345.
- [9] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
- [10] Henggang Cui, James Cipar, Qirong Ho, Jin Kyu Kim, Seunghak Lee, Abhimanu Kumar, Jinliang Wei, Wei Dai, Gregory R Ganger, Phillip B Gibbons, et al. 2014. Exploiting bounded staleness to speed up big data analytics. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)*. 37–48.
- [11] Wei Dai, Abhimanu Kumar, Jinliang Wei, Qirong Ho, Garth Gibson, and Eric Xing. 2015. High-performance distributed ML at scale through parameter server consistency models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 29.
- [12] Jeffrey Dean, Greg S Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Quoc V Le, Mark Z Mao, Marc'Aurelio Ranzato, Andrew Senior, Paul Tucker, et al. 2012. Large scale distributed deep networks. (2012).
- [13] Shiqing Fan, Yi Rong, Chen Meng, Zongyan Cao, Siyu Wang, Zhen Zheng, Chuan Wu, Guoping Long, Jun Yang, Lixue Xia, et al. 2021. DAPPLE: a pipelined data parallel approach for training large models. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 431–445.
- [14] Jiarui Fang, Haohuan Fu, Guangwen Yang, and Cho-Jui Hsieh. 2019. RedSync: reducing synchronization bandwidth for distributed deep learning training system. *J. Parallel and Distrib. Comput.* 133 (2019), 30–39.
- [15] Alexandros V Gerbessiotis and Leslie G Valiant. 1994. Direct bulk-synchronous parallel algorithms. *Journal of parallel and distributed computing* 22, 2 (1994), 251–267.
- [16] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyröla, Andrew Tulloch, Yangqing Jia, and Kaiming He. 2017. Accurate, large minibatch sgd: Training imagenet in 1 hour. *arXiv preprint arXiv:1706.02677* (2017).
- [17] Andreas Griewank and Andrea Walther. 2000. Algorithm 799: revolve: an implementation of checkpointing for the reverse or adjoint mode of computational differentiation. *ACM Transactions on Mathematical Software (TOMS)* 26, 1 (2000), 19–45.
- [18] Qirong Ho, James Cipar, Henggang Cui, Jin Kyu Kim, Seunghak Lee, Phillip B Gibbons, Garth A Gibson, Gregory R Ganger, and Eric P Xing. 2013. More effective distributed ml via a stale synchronous parallel parameter server. *Advances in neural information processing systems* 2013 (2013), 1223.
- [19] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2018.

- Gpipe: Efficient training of giant neural networks using pipeline parallelism. *arXiv preprint arXiv:1811.06965* (2018).
- [20] Anand Jayarajan, Jinliang Wei, Garth Gibson, Alexandra Fedorova, and Gennady Pekhimenko. 2019. Priority-based parameter propagation for distributed DNN training. *arXiv preprint arXiv:1905.03960* (2019).
 - [21] Zhihao Jia, Matei Zaharia, and Alex Aiken. 2018. Beyond Data and Model Parallelism for Deep Neural Networks. *CoRR abs/1807.05358* (2018). [arXiv:1807.05358](https://arxiv.org/abs/1807.05358) <http://arxiv.org/abs/1807.05358>
 - [22] Jin Kyu Kim, Qirong Ho, Seunghak Lee, Xun Zheng, Wei Dai, Garth A. Gibson, and Eric P. Xing. 2016. STRADS: A Distributed Framework for Scheduled Model Parallel Machine Learning. In *Proceedings of the Eleventh European Conference on Computer Systems* (London, United Kingdom) (*EuroSys '16*). Association for Computing Machinery, New York, NY, USA, Article 5, 16 pages. <https://doi.org/10.1145/2901318.2901331>
 - [23] Soojeong Kim, Gyeong-In Yu, Hoin Park, Sungwoo Cho, Eunji Jeong, Hyeonmin Ha, Sanha Lee, Joo Seong Jeong, and Byung-Gon Chun. 2019. Parallax: Sparsity-aware data parallel training of deep neural networks. In *Proceedings of the Fourteenth EuroSys Conference 2019*. 1–15.
 - [24] Alex Krizhevsky. 2014. One weird trick for parallelizing convolutional neural networks. *CoRR abs/1404.5997* (2014). [arXiv:1404.5997](https://arxiv.org/abs/1404.5997) <http://arxiv.org/abs/1404.5997>
 - [25] Seunghak Lee, Jin Kyu Kim, Xun Zheng, Qirong Ho, Garth Gibson, and Eric P Xing. 2014. On Model Parallelization and Scheduling Strategies for Distributed Machine Learning. (12 2014). <https://doi.org/10.1184/R1/6476048.v1>
 - [26] Mu Li, David G. Andersen, Jun Woo Park, Alexander J. Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J. Shekita, and Bor-Ying Su. 2014. Scaling Distributed Machine Learning with the Parameter Server. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, Broomfield, CO, 583–598. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/li_mu
 - [27] Youjie Li, Mingchao Yu, Songze Li, Salman Avestimehr, Nam Sung Kim, and Alexander Schwing. 2018. Pipe-sgd: A decentralized pipelined sgd framework for distributed deep net training. *arXiv preprint arXiv:1811.03619* (2018).
 - [28] Xiangru Lian, Wei Zhang, Ce Zhang, and Ji Liu. 2018. Asynchronous decentralized parallel stochastic gradient descent. In *International Conference on Machine Learning*. PMLR, 3043–3052.
 - [29] Xiangrui Meng, Joseph Bradley, Burak Yavuz, Evan Sparks, Shivaram Venkataraman, Davies Liu, Jeremy Freeman, DB Tsai, Manish Amde, Sean Owen, et al. 2016. Mllib: Machine learning in apache spark. *The Journal of Machine Learning Research* 17, 1 (2016), 1235–1241.
 - [30] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized Pipeline Parallelism for DNN Training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles* (Huntsville, Ontario, Canada) (*SOSP '19*). Association for Computing Machinery, New York, NY, USA, 1–15. <https://doi.org/10.1145/3341301.3359646>
 - [31] Deepak Narayanan, Amar Phanishayee, Kaiyu Shi, Xie Chen, and Matei Zaharia. 2020. Memory-efficient pipeline-parallel dnn training. *arXiv preprint arXiv:2006.09503* (2020).
 - [32] Feng Niu, Benjamin Recht, Christopher Ré, and Stephen J Wright. 2011. Hogwild!: A lock-free approach to parallelizing stochastic gradient descent. *arXiv preprint arXiv:1106.5730* (2011).
 - [33] Chanyoung Oh, Zhen Zheng, Xipeng Shen, Jidong Zhai, and Youngmin Yi. 2020. GOpPipe: a granularity-oblivious programming framework for pipelined stencil executions on GPU. In *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques*. 43–54.
 - [34] Saptadeep Pal, Eiman Ebrahimi, Arslan Zulfiqar, Yaosheng Fu, Victor Zhang, Szymon Migacz, David Nellans, and Puneet Gupta. 2019. Optimizing multi-GPU parallelization strategies for deep learning training. *IEEE Micro* 39, 5 (2019), 91–101.
 - [35] Jay H Park, Gyeongchan Yun, M Yi Chang, Nguyen T Nguyen, Seungmin Lee, Jaesik Choi, Sam H Noh, and Young-ri Choi. 2020. HetPipe: Enabling Large DNN Training on (Whimpy) Heterogeneous GPU Clusters through Integration of Pipelined Model Parallelism and Data Parallelism. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. 307–321.
 - [36] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. 2017. Automatic differentiation in pytorch. (2017).
 - [37] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding by generative pre-training. (2018).
 - [38] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
 - [39] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V Le. 2019. Regularized evolution for image classifier architecture search. In *Proceedings of the aaai conference on artificial intelligence*, Vol. 33. 4780–4789.
 - [40] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. 2015. Imagenet large scale visual recognition challenge. *International journal of computer vision* 115, 3 (2015), 211–252.
 - [41] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Edinburgh neural machine translation systems for wmt 16. *arXiv preprint arXiv:1606.02891* (2016).
 - [42] Alexander Sergeev and Mike Del Balso. 2018. Horovod: fast and easy distributed deep learning in TensorFlow. *CoRR abs/1802.05799* (2018). [arXiv:1802.05799](https://arxiv.org/abs/1802.05799) <http://arxiv.org/abs/1802.05799>
 - [43] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014).
 - [44] Jizhe Wang, Pipei Huang, Huan Zhao, Zhibo Zhang, Binqiang Zhao, and Dik Lun Lee. 2018. Billion-scale commodity embedding for e-commerce recommendation in alibaba. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 839–848.
 - [45] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. 2018. Superneurons: Dynamic GPU memory management for training deep neural networks. In *Proceedings of the 23rd ACM SIGPLAN symposium on principles and practice of parallel programming*. 41–53.
 - [46] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. 2016. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144* (2016).
 - [47] Bowen Yang, Jian Zhang, Jonathan Li, Christopher Ré, Christopher Aberger, and Christopher De Sa. 2021. Pipemare: Asynchronous pipeline parallel dnn training. *Proceedings of Machine Learning and Systems* 3 (2021).
 - [48] Jun Zhan and Jinghui Zhang. 2019. Pipe-torch: Pipeline-based distributed deep learning in a gpu cluster with heterogeneous networking. In *2019 Seventh International Conference on Advanced Cloud and Big Data (CBD)*. IEEE, 55–60.
 - [49] Hao Zhang, Zeyu Zheng, Shizhen Xu, Wei Dai, Qirong Ho, Xiaodan Liang, Zhiting Hu, Jinliang Wei, Pengtao Xie, and Eric P Xing. 2017. Poseidon: An efficient communication architecture for distributed deep learning on GPU clusters. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. 181–193.
 - [50] Zhen Zheng, Chanyoung Oh, Jidong Zhai, Xipeng Shen, Youngmin Yi, and Wenguang Chen. 2017. Versapipe: a versatile programming framework for pipelined computing on GPU. In *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 587–599.